

4.0 Introduction

Numerical integration, which is also called *quadrature*, has a history extending back to the invention of calculus and before. The fact that integrals of elementary functions could not, in general, be computed analytically, while derivatives *could* be, served to give the field a certain panache, and to set it a cut above the arithmetic drudgery of numerical analysis during the whole of the 18th and 19th centuries.

With the invention of automatic computing, quadrature became just one numerical task among many, and not a very interesting one at that. Automatic computing, even the most primitive sort involving desk calculators and rooms full of “computers” (that were, until the 1950s, people rather than machines), opened to feasibility the much richer field of numerical integration of differential equations. Quadrature is merely the simplest special case: The evaluation of the integral

$$I = \int_a^b f(x)dx \quad (4.0.1)$$

is precisely equivalent to solving for the value $I \equiv y(b)$ the differential equation

$$\frac{dy}{dx} = f(x) \quad (4.0.2)$$

with the boundary condition

$$y(a) = 0 \quad (4.0.3)$$

Chapter 17 of this book deals with the numerical integration of differential equations. In that chapter, much emphasis is given to the concept of “variable” or “adaptive” choices of stepsize. We will not, therefore, develop that material here. If the function that you propose to integrate is sharply concentrated in one or more peaks, or if its shape is not readily characterized by a single length scale, then it is likely that you should cast the problem in the form of (4.0.2) – (4.0.3) and use the methods of Chapter 17. (But take a look at §4.7 first.)

The quadrature methods in this chapter are based, in one way or another, on the obvious device of adding up the value of the integrand at a sequence of abscissas

within the range of integration. The game is to obtain the integral as accurately as possible with the smallest number of function evaluations of the integrand. Just as in the case of interpolation (Chapter 3), one has the freedom to choose methods of various *orders*, with higher order sometimes, but not always, giving higher accuracy. *Romberg integration*, which is discussed in §4.3, is a general formalism for making use of integration methods of a variety of different orders, and we recommend it highly.

Apart from the methods of this chapter and of Chapter 17, there are yet other methods for obtaining integrals. One important class is based on function approximation. We discuss explicitly the integration of functions by Chebyshev approximation (*Clenshaw-Curtis quadrature*) in §5.9. Although not explicitly discussed here, you ought to be able to figure out how to do *cubic spline quadrature* using the output of the routine `spline` in §3.3. (Hint: Integrate equation 3.3.3 over x analytically. See [1].)

Some integrals related to Fourier transforms can be calculated using the fast Fourier transform (FFT) algorithm. This is discussed in §13.9. A related problem is the evaluation of integrals with long oscillatory tails. This is discussed at the end of §5.3.

Multidimensional integrals are a whole 'nother multidimensional bag of worms. Section 4.8 is an introductory discussion in this chapter; the important technique of *Monte Carlo integration* is treated in Chapter 7.

CITED REFERENCES AND FURTHER READING:

- Carnahan, B., Luther, H.A., and Wilkes, J.O. 1969, *Applied Numerical Methods* (New York: Wiley), Chapter 2.
- Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), Chapter 7.
- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 4.
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 3.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), Chapter 4.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.4.
- Kahaner, D., Moler, C., and Nash, S. 1989, *Numerical Methods and Software* (Englewood Cliffs, NJ: Prentice-Hall), Chapter 5.
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §5.2, p. 89.[1]
- Davis, P., and Rabinowitz, P. 1984, *Methods of Numerical Integration*, 2nd ed. (Orlando, FL: Academic Press).

4.1 Classical Formulas for Equally Spaced Abscissas

Where would any book on numerical analysis be without Mr. Simpson and his “rule”? The classical formulas for integrating a function whose value is known at

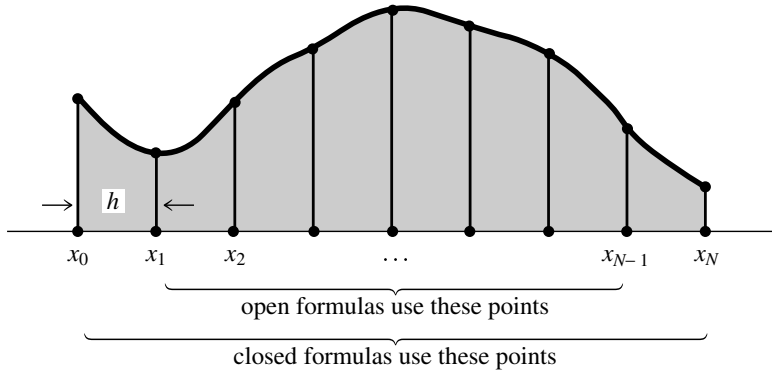


Figure 4.1.1. Quadrature formulas with equally spaced abscissas compute the integral of a function between x_0 and x_N . Closed formulas evaluate the function on the boundary points, while open formulas refrain from doing so (useful if the evaluation algorithm breaks down on the boundary points).

equally spaced steps have a certain elegance about them, and they are redolent with historical association. Through them, the modern numerical analyst communes with the spirits of his or her predecessors back across the centuries, as far as the time of Newton, if not farther. Alas, times *do* change; with the exception of two of the most modest formulas (“extended trapezoidal rule,” equation 4.1.11, and “extended midpoint rule,” equation 4.1.19; see §4.2), the classical formulas are almost entirely useless. They are museum pieces, but beautiful ones; we now enter the museum. (You can skip to §4.2 if you are not touristically inclined.)

Some notation: We have a sequence of abscissas, denoted $x_0, x_1, \dots, x_{N-1}, x_N$, that are spaced apart by a constant step h ,

$$x_i = x_0 + ih \quad i = 0, 1, \dots, N \quad (4.1.1)$$

A function $f(x)$ has known values at the x_i ’s,

$$f(x_i) \equiv f_i \quad (4.1.2)$$

We want to integrate the function $f(x)$ between a lower limit a and an upper limit b , where a and b are each equal to one or the other of the x_i ’s. An integration formula that uses the value of the function at the endpoints, $f(a)$ or $f(b)$, is called a *closed* formula. Occasionally, we want to integrate a function whose value at one or both endpoints is difficult to compute (e.g., the computation of f goes to a limit of zero over zero there, or worse yet has an integrable singularity there). In this case we want an *open* formula, which estimates the integral using only x_i ’s strictly *between* a and b (see Figure 4.1.1).

The basic building blocks of the classical formulas are rules for integrating a function over a small number of intervals. As that number increases, we can find rules that are exact for polynomials of increasingly high order. (Keep in mind that higher order does not always imply higher accuracy in real cases.) A sequence of such closed formulas is now given.

4.1.1 Closed Newton-Cotes Formulas

Trapezoidal rule:

$$\int_{x_0}^{x_1} f(x)dx = h \left[\frac{1}{2} f_0 + \frac{1}{2} f_1 \right] + O(h^3 f'') \quad (4.1.3)$$

Here the error term $O(\)$ signifies that the true answer differs from the estimate by an amount that is the product of some numerical coefficient times h^3 times the value of the function's second derivative somewhere in the interval of integration. The coefficient is knowable, and it can be found in all the standard references on this subject. The point at which the second derivative is to be evaluated is, however, unknowable. If we knew it, we could evaluate the function there and have a higher-order method! Since the product of a knowable and an unknowable is unknowable, we will streamline our formulas and write only $O(\)$, instead of the coefficient.

Equation (4.1.3) is a two-point formula (x_0 and x_1). It is exact for polynomials up to and including degree 1, i.e., $f(x) = x$. One anticipates that there is a three-point formula exact up to polynomials of degree 2. This is true; moreover, by a cancellation of coefficients due to left-right symmetry of the formula, the three-point formula is exact for polynomials up to and including degree 3, i.e., $f(x) = x^3$.

Simpson's rule:

$$\int_{x_0}^{x_2} f(x)dx = h \left[\frac{1}{3} f_0 + \frac{4}{3} f_1 + \frac{1}{3} f_2 \right] + O(h^5 f^{(4)}) \quad (4.1.4)$$

Here $f^{(4)}$ means the fourth derivative of the function f evaluated at an unknown place in the interval. Note also that the formula gives the integral over an interval of size $2h$, so the coefficients add up to 2.

There is no lucky cancellation in the four-point formula, so it is also exact for polynomials up to and including degree 3.

Simpson's $\frac{3}{8}$ rule:

$$\int_{x_0}^{x_3} f(x)dx = h \left[\frac{3}{8} f_0 + \frac{9}{8} f_1 + \frac{9}{8} f_2 + \frac{3}{8} f_3 \right] + O(h^5 f^{(4)}) \quad (4.1.5)$$

The five-point formula again benefits from a cancellation:

Bode's rule:

$$\int_{x_0}^{x_4} f(x)dx = h \left[\frac{14}{45} f_0 + \frac{64}{45} f_1 + \frac{24}{45} f_2 + \frac{64}{45} f_3 + \frac{14}{45} f_4 \right] + O(h^7 f^{(6)}) \quad (4.1.6)$$

This is exact for polynomials up to and including degree 5.

At this point the formulas stop being named after famous personages, so we will not go any further. Consult [1] for additional formulas in the sequence.

4.1.2 Extrapolative Formulas for a Single Interval

We are going to depart from historical practice for a moment. Many texts would give, at this point, a sequence of "Newton-Cotes Formulas of Open Type." Here is

an example:

$$\int_{x_0}^{x_5} f(x)dx = h \left[\frac{55}{24} f_1 + \frac{5}{24} f_2 + \frac{5}{24} f_3 + \frac{55}{24} f_4 \right] + O(h^5 f^{(4)})$$

Notice that the integral from $a = x_0$ to $b = x_5$ is estimated, using only the interior points x_1, x_2, x_3, x_4 . In our opinion, formulas of this type are not useful for the reasons that (i) they cannot usefully be strung together to get “extended” rules, as we are about to do with the closed formulas, and (ii) for all other possible uses they are dominated by the Gaussian integration formulas, which we will introduce in §4.6.

Instead of the Newton-Cotes open formulas, let us set out the formulas for estimating the integral in the single interval from x_0 to x_1 , using values of the function f at $x_1, x_2, \dots$. These will be useful building blocks later for the “extended” open formulas.

$$\int_{x_0}^{x_1} f(x)dx = h[f_1] + O(h^2 f') \quad (4.1.7)$$

$$\int_{x_0}^{x_1} f(x)dx = h \left[\frac{3}{2} f_1 - \frac{1}{2} f_2 \right] + O(h^3 f'') \quad (4.1.8)$$

$$\int_{x_0}^{x_1} f(x)dx = h \left[\frac{23}{12} f_1 - \frac{16}{12} f_2 + \frac{5}{12} f_3 \right] + O(h^4 f^{(3)}) \quad (4.1.9)$$

$$\int_{x_0}^{x_1} f(x)dx = h \left[\frac{55}{24} f_1 - \frac{59}{24} f_2 + \frac{37}{24} f_3 - \frac{9}{24} f_4 \right] + O(h^5 f^{(4)}) \quad (4.1.10)$$

Perhaps a word here would be in order about how formulas like the above can be derived. There are elegant ways, but the most straightforward is to write down the basic form of the formula, replacing the numerical coefficients with unknowns, say p, q, r, s . Without loss of generality take $x_0 = 0$ and $x_1 = 1$, so $h = 1$. Substitute in turn for $f(x)$ (and for f_1, f_2, f_3, f_4) the functions $f(x) = 1, f(x) = x, f(x) = x^2$, and $f(x) = x^3$. Doing the integral in each case reduces the left-hand side to a number and the right-hand side to a linear equation for the unknowns p, q, r, s . Solving the four equations produced in this way gives the coefficients.

4.1.3 Extended Formulas (Closed)

If we use equation (4.1.3) $N - 1$ times to do the integration in the intervals $(x_0, x_1), (x_1, x_2), \dots, (x_{N-2}, x_{N-1})$ and then add the results, we obtain an “extended” or “composite” formula for the integral from x_0 to x_{N-1} .

Extended trapezoidal rule:

$$\int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{1}{2} f_0 + f_1 + f_2 + \dots + f_{N-2} + \frac{1}{2} f_{N-1} \right] + O \left(\frac{(b-a)^3 f''}{N^2} \right) \quad (4.1.11)$$

Here we have written the error estimate in terms of the interval $b - a$ and the number of points N instead of in terms of h . This is clearer, since one is usually holding a and

b fixed and wanting to know, e.g., how much the error will be decreased by taking twice as many steps (in this case, it is by a factor of 4). In subsequent equations we will show *only* the scaling of the error term with the number of steps.

For reasons that will not become clear until §4.2, equation (4.1.11) is in fact the most important equation in this section; it is the basis for most practical quadrature schemes.

The *extended formula of order $1/N^3$* is

$$\begin{aligned} \int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{5}{12}f_0 + \frac{13}{12}f_1 + f_2 + f_3 + \right. \\ \left. \cdots + f_{N-3} + \frac{13}{12}f_{N-2} + \frac{5}{12}f_{N-1} \right] + O\left(\frac{1}{N^3}\right) \end{aligned} \quad (4.1.12)$$

(We will see in a moment where this comes from.)

If we apply equation (4.1.4) to successive, nonoverlapping *pairs* of intervals, we get the *extended Simpson's rule*:

$$\begin{aligned} \int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{1}{3}f_0 + \frac{4}{3}f_1 + \frac{2}{3}f_2 + \frac{4}{3}f_3 + \right. \\ \left. \cdots + \frac{2}{3}f_{N-3} + \frac{4}{3}f_{N-2} + \frac{1}{3}f_{N-1} \right] + O\left(\frac{1}{N^4}\right) \end{aligned} \quad (4.1.13)$$

Notice that the $2/3$, $4/3$ alternation continues throughout the interior of the evaluation. Many people believe that the wobbling alternation somehow contains deep information about the integral of their function that is not apparent to mortal eyes. In fact, the alternation is an artifact of using the building block (4.1.4). Another extended formula with the same order as Simpson's rule is

$$\begin{aligned} \int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{3}{8}f_0 + \frac{7}{6}f_1 + \frac{23}{24}f_2 + f_3 + f_4 + \right. \\ \left. \cdots + f_{N-5} + f_{N-4} + \frac{23}{24}f_{N-3} + \frac{7}{6}f_{N-2} + \frac{3}{8}f_{N-1} \right] \\ + O\left(\frac{1}{N^4}\right) \end{aligned} \quad (4.1.14)$$

This equation is constructed by fitting cubic polynomials through successive groups of four points; we defer details to §19.3, where a similar technique is used in the solution of integral equations. We can, however, tell you where equation (4.1.12) came from. It is Simpson's extended rule, averaged with a modified version of itself in which the first and last steps are done with the trapezoidal rule (4.1.3). The trapezoidal step is *two* orders lower than Simpson's rule; however, its contribution to the integral goes down as an additional power of N (since it is used only twice, not N times). This makes the resulting formula of degree *one* less than Simpson.

4.1.4 Extended Formulas (Open and Semi-Open)

We can construct open and semi-open extended formulas by adding the closed formulas (4.1.11) – (4.1.14), evaluated for the second and subsequent steps, to the

extrapolative open formulas for the first step, (4.1.7) – (4.1.10). As discussed immediately above, it is consistent to use an end step that is of one order lower than the (repeated) interior step. The resulting formulas for an interval open at both ends are as follows.

Equations (4.1.7) and (4.1.11) give

$$\int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{3}{2}f_1 + f_2 + f_3 + \cdots + f_{N-3} + \frac{3}{2}f_{N-2} \right] + O\left(\frac{1}{N^2}\right) \quad (4.1.15)$$

Equations (4.1.8) and (4.1.12) give

$$\int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{23}{12}f_1 + \frac{7}{12}f_2 + f_3 + f_4 + \cdots + f_{N-4} + \frac{7}{12}f_{N-3} + \frac{23}{12}f_{N-2} \right] + O\left(\frac{1}{N^3}\right) \quad (4.1.16)$$

Equations (4.1.9) and (4.1.13) give

$$\int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{27}{12}f_1 + 0 + \frac{13}{12}f_3 + \frac{4}{3}f_4 + \cdots + \frac{4}{3}f_{N-5} + \frac{13}{12}f_{N-4} + 0 + \frac{27}{12}f_{N-2} \right] + O\left(\frac{1}{N^4}\right) \quad (4.1.17)$$

The interior points alternate 4/3 and 2/3. If we want to avoid this alternation, we can combine equations (4.1.9) and (4.1.14), giving

$$\int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{55}{24}f_1 - \frac{1}{6}f_2 + \frac{11}{8}f_3 + f_4 + f_5 + f_6 + \cdots + f_{N-6} + f_{N-5} + \frac{11}{8}f_{N-4} - \frac{1}{6}f_{N-3} + \frac{55}{24}f_{N-2} \right] + O\left(\frac{1}{N^4}\right) \quad (4.1.18)$$

We should mention in passing another extended open formula, for use where the limits of integration are located halfway between tabulated abscissas. This one is known as the *extended midpoint rule* and is accurate to the same order as (4.1.15):

$$\int_{x_0}^{x_{N-1}} f(x)dx = h[f_{1/2} + f_{3/2} + f_{5/2} + \cdots + f_{N-5/2} + f_{N-3/2}] + O\left(\frac{1}{N^2}\right) \quad (4.1.19)$$

There are also formulas of higher order for this situation, but we will refrain from giving them.

The *semi-open formulas* are just the obvious combinations of equations (4.1.11) – (4.1.14) with (4.1.15) – (4.1.18), respectively. At the closed end of the integration, use the weights from the former equations; at the open end, use the weights from

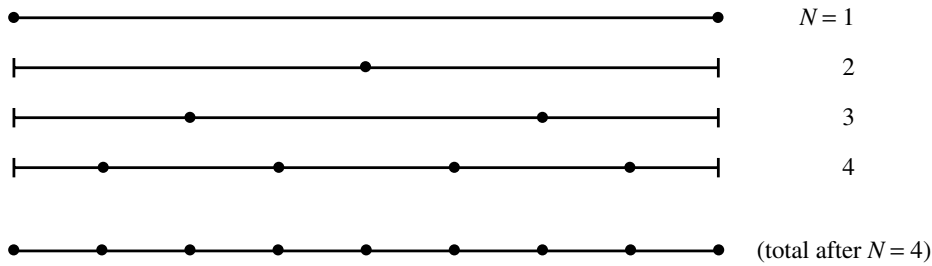


Figure 4.2.1. Sequential calls to the routine `Trapzd` incorporate the information from previous calls and evaluate the integrand only at those new points necessary to refine the grid. The bottom line shows the totality of function evaluations after the fourth call. The routine `qsimp`, by weighting the intermediate results, transforms the trapezoid rule into Simpson’s rule with essentially no additional overhead.

the latter equations. One example should give the idea, the formula with error term decreasing as $1/N^3$, which is closed on the right and open on the left:

$$\int_{x_0}^{x_{N-1}} f(x)dx = h \left[\frac{23}{12}f_1 + \frac{7}{12}f_2 + f_3 + f_4 + \cdots + f_{N-3} + \frac{13}{12}f_{N-2} + \frac{5}{12}f_{N-1} \right] + O\left(\frac{1}{N^3}\right) \quad (4.1.20)$$

CITED REFERENCES AND FURTHER READING:

- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nrl.com/aands>, §25.4.[1]
- Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), §7.1.

4.2 Elementary Algorithms

Our starting point is equation (4.1.11), the extended trapezoidal rule. There are two facts about the trapezoidal rule that make it the starting point for a variety of algorithms. One fact is rather obvious, while the second is rather “deep.”

The obvious fact is that, for a fixed function $f(x)$ to be integrated between fixed limits a and b , one can double the number of intervals in the extended trapezoidal rule without losing the benefit of previous work. The coarsest implementation of the trapezoidal rule is to average the function at its endpoints a and b . The first stage of refinement is to add to this average the value of the function at the halfway point. The second stage of refinement is to add the values at the $1/4$ and $3/4$ points. And so on (see Figure 4.2.1).

As we will see, a number of elementary quadrature algorithms involve adding successive stages of refinement. It is convenient to encapsulate this feature in a `Quadrature` structure:


```
struct Quadrature{
```

quadrature.h

Abstract base class for elementary quadrature algorithms.

```
    Int n;                                Current level of refinement.
```

```
    virtual Doub next() = 0;
```

Returns the value of the integral at the nth stage of refinement. The function next() must be defined in the derived class.

```
};
```

Then the Trapzd structure is derived from this as follows:

```
template<class T>
```

quadrature.h

```
struct Trapzd : Quadrature {
```

Routine implementing the extended trapezoidal rule.

```
    Doub a,b,s;                            Limits of integration and current value of integral.
```

```
    T &func;
```

```
    Trapzd() {};
```

```
    Trapzd(T &funcc, const Doub aa, const Doub bb) :
```

```
        func(funcc), a(aa), b(bb) {n=0;}
```

The constructor takes as inputs func, the function or functor to be integrated between limits a and b, also input.

```
    Doub next() {
```

Returns the nth stage of refinement of the extended trapezoidal rule. On the first call (n=1), the routine returns the crudest estimate of $\int_a^b f(x)dx$. Subsequent calls set n=2,3,... and improve the accuracy by adding 2^{n-2} additional interior points.

```
        Doub x,tnm,sum,del;
```

```
        Int it,j;
```

```
        n++;
```

```
        if (n == 1) {
```

```
            return (s=0.5*(b-a)*(func(a)+func(b)));
```

```
        } else {
```

```
            for (it=1,j=1;j<n-1;j++) it <= 1;
```

```
            tnm=it;
```

```
            del=(b-a)/tnm;
```

This is the spacing of the points to be added.

```
            x=a+0.5*del;
```

```
            for (sum=0.0,j=0;j<it;j++,x+=del) sum += func(x);
```

```
            s=0.5*(s+(b-a)*sum/tnm);
```

This replaces s by its refined value.

```
            return s;
```

```
        }
```

```
    }
```

```
};
```

Note that Trapzd is templated on the whole struct and does not just contain a templated function. This is necessary because it retains a reference to the supplied function or functor as a member variable.

The Trapzd structure is a workhorse that can be harnessed in several ways. The simplest and crudest is to integrate a function by the extended trapezoidal rule where you know in advance (we can't imagine how!) the number of steps you want. If you want $2^M + 1$, you can accomplish this by the fragment

```
Ftor func;
```

Functor func here has no parameters.

```
Trapzd<Ftor> s(func,a,b);
```

```
for(j=1;j<=m+1;j++) val=s.next();
```

with the answer returned as val. Here Ftor is a functor containing the function to be integrated.

Much better, of course, is to refine the trapezoidal rule until some specified degree of accuracy has been achieved. A function for this is

quadrature.h

```

template<class T>
Doub qtrap(T &func, const Doub a, const Doub b, const Doub eps=1.0e-10) {
Returns the integral of the function or functor func from a to b. The constants EPS can be
set to the desired fractional accuracy and JMAX so that 2 to the power JMAX-1 is the maximum
allowed number of steps. Integration is performed by the trapezoidal rule.
    const Int JMAX=20;
    Doub s,olds=0.0;           Initial value of olds is arbitrary.
    Trapzd<T> t(func,a,b);
    for (Int j=0;j<JMAX;j++) {
        s=t.next();
        if (j > 5)             Avoid spurious early convergence.
            if (abs(s-olds) < eps*abs(olds) ||
                (s == 0.0 && olds == 0.0)) return s;
        olds=s;
    }
    throw("Too many steps in routine qtrap");
}

```

The optional argument `eps` sets the desired fractional accuracy. Unsophisticated as it is, routine `qtrap` is in fact a fairly robust way of doing integrals of functions that are not very smooth. Increased sophistication will usually translate into a higher-order method whose efficiency will be greater only for sufficiently smooth integrands. `qtrap` is the method of choice, e.g., for an integrand that is a function of a variable that is linearly interpolated between measured data points. Be sure that you do not require too stringent an `eps`, however: If `qtrap` takes too many steps in trying to achieve your required accuracy, accumulated roundoff errors may start increasing, and the routine may never converge. A value of 10^{-10} or even smaller is usually no problem in double precision when the convergence is moderately rapid, but not otherwise. (Of course, very few problems really require such precision.)

We come now to the “deep” fact about the extended trapezoidal rule, equation (4.1.11). It is this: The error of the approximation, which begins with a term of order $1/N^2$, is in fact *entirely even* when expressed in powers of $1/N$. This follows directly from the *Euler-Maclaurin summation formula*,

$$\begin{aligned}
 \int_{x_0}^{x_{N-1}} f(x)dx &= h \left[\frac{1}{2} f_0 + f_1 + f_2 + \cdots + f_{N-2} + \frac{1}{2} f_{N-1} \right] \\
 &\quad - \frac{B_2 h^2}{2!} (f'_{N-1} - f'_0) - \cdots - \frac{B_{2k} h^{2k}}{(2k)!} (f^{(2k-1)}_{N-1} - f^{(2k-1)}_0) - \cdots
 \end{aligned}
 \tag{4.2.1}$$

Here B_{2k} is a *Bernoulli number*, defined by the generating function

$$\frac{t}{e^t - 1} = \sum_{n=0}^{\infty} B_n \frac{t^n}{n!}
 \tag{4.2.2}$$

with the first few even values (odd values vanish except for $B_1 = -1/2$)

$$\begin{aligned}
 B_0 &= 1 & B_2 &= \frac{1}{6} & B_4 &= -\frac{1}{30} & B_6 &= \frac{1}{42} \\
 B_8 &= -\frac{1}{30} & B_{10} &= \frac{5}{66} & B_{12} &= -\frac{691}{2730}
 \end{aligned}
 \tag{4.2.3}$$

Equation (4.2.1) is not a convergent expansion, but rather only an asymptotic expansion whose error when truncated at any point is always less than twice the magnitude

of the first neglected term. The reason that it is not convergent is that the Bernoulli numbers become very large, e.g.,

$$B_{50} = \frac{495057205241079648212477525}{66}$$

The key point is that only even powers of h occur in the error series of (4.2.1). This fact is not, in general, shared by the higher-order quadrature rules in §4.1. For example, equation (4.1.12) has an error series beginning with $O(1/N^3)$, but continuing with all subsequent powers of N : $1/N^4$, $1/N^5$, etc.

Suppose we evaluate (4.1.11) with N steps, getting a result S_N , and then again with $2N$ steps, getting a result S_{2N} . (This is done by any two consecutive calls of `Trapzd`.) The leading error term in the second evaluation will be $1/4$ the size of the error in the first evaluation. Therefore the combination

$$S = \frac{4}{3}S_{2N} - \frac{1}{3}S_N \quad (4.2.4)$$

will cancel out the leading order error term. But there *is* no error term of order $1/N^3$, by (4.2.1). The surviving error is of order $1/N^4$, the same as Simpson's rule. In fact, it should not take long for you to see that (4.2.4) is *exactly* Simpson's rule (4.1.13), alternating $2/3$'s, $4/3$'s, and all. This is the preferred method for evaluating that rule, and we can write it as a routine exactly analogous to `qtrap` above:

```
template<class T>
Doub qsimp(T &func, const Doub a, const Doub b, const Doub eps=1.0e-10) {
Returns the integral of the function or functor func from a to b. The constants EPS can be
set to the desired fractional accuracy and JMAX so that 2 to the power JMAX-1 is the maximum
allowed number of steps. Integration is performed by Simpson's rule.
    const Int JMAX=20;
    Doub s,st,ost=0.0,os=0.0;
    Trapzd<T> t(func,a,b);
    for (Int j=0;j<JMAX;j++) {
        st=t.next();
        s=(4.0*st-ost)/3.0;           Compare equation (4.2.4), above.
        if (j > 5)                   Avoid spurious early convergence.
            if (abs(s-os) < eps*abs(os) ||
                (s == 0.0 && os == 0.0)) return s;
        os=s;
        ost=st;
    }
    throw("Too many steps in routine qsimp");
}
```

quadrature.h

The routine `qsimp` will in general be more efficient than `qtrap` (i.e., require fewer function evaluations) when the function to be integrated has a finite fourth derivative (i.e., a continuous third derivative). The combination of `qsimp` and its necessary workhorse `Trapzd` is a good one for light-duty work.

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §3.1.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.4.1 – §7.4.2.
- Forsythe, G.E., Malcolm, M.A., and Moler, C.B. 1977, *Computer Methods for Mathematical Computations* (Englewood Cliffs, NJ: Prentice-Hall), §5.3.

4.3 Romberg Integration

We can view Romberg's method as the natural generalization of the routine `qsimp` in the last section to integration schemes that are of higher order than Simpson's rule. The basic idea is to use the results from k successive refinements of the extended trapezoidal rule (implemented in `trapzd`) to remove all terms in the error series up to but not including $O(1/N^{2k})$. The routine `qsimp` is the case of $k = 2$. This is one example of a very general idea that goes by the name of *Richardson's deferred approach to the limit*: Perform some numerical algorithm for various values of a parameter h , and then extrapolate the result to the continuum limit $h = 0$.

Equation (4.2.4), which subtracts off the leading error term, is a special case of polynomial extrapolation. In the more general Romberg case, we can use Neville's algorithm (see §3.2) to extrapolate the successive refinements to zero stepsize. Neville's algorithm can in fact be coded very concisely within a Romberg integration routine. For clarity of the program, however, it seems better to do the extrapolation by a function call to `Poly_interp::rawinterp`, as given in §3.2.

```
romberg.h  template <class T>
           Doub qromb(T &func, Doub a, Doub b, const Doub eps=1.0e-10) {
Returns the integral of the function or functor func from a to b. Integration is performed by
Romberg's method of order 2K, where, e.g., K=2 is Simpson's rule.
           const Int JMAX=20, JMAXP=JMAX+1, K=5;
           Here EPS is the fractional accuracy desired, as determined by the extrapolation error estimate;
           JMAX limits the total number of steps; K is the number of points used in the extrapolation.
           VecDoub s(JMAX),h(JMAXP);           These store the successive trapezoidal approxi-
           Poly_interp polint(h,s,K);           mations and their relative stepsizes.
           h[0]=1.0;
           Trapzd<T> t(func,a,b);
           for (Int j=1;j<=JMAX;j++) {
               s[j-1]=t.next();
               if (j >= K) {
                   Doub ss=polint.rawinterp(j-K,0.0);
                   if (abs(polint.dy) <= eps*abs(ss)) return ss;
               }
               h[j]=0.25*h[j-1];
               This is a key step: The factor is 0.25 even though the stepsize is decreased by only
               0.5. This makes the extrapolation a polynomial in  $h^2$  as allowed by equation (4.2.1),
               not just a polynomial in  $h$ .
           }
           throw("Too many steps in routine qromb");
}
```

The routine `qromb` is quite powerful for sufficiently smooth (e.g., analytic) integrands, integrated over intervals that contain no singularities, and where the end-points are also nonsingular. `qromb`, in such circumstances, takes many, *many* fewer function evaluations than either of the routines in §4.2. For example, the integral

$$\int_0^2 x^4 \log(x + \sqrt{x^2 + 1}) dx$$

converges (with parameters as shown above) on the second extrapolation, after just 6 calls to `trapzd`, while `qsimp` requires 11 calls (32 times as many evaluations of the integrand) and `qtrap` requires 19 calls (8192 times as many evaluations of the integrand).

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §3.4 – §3.5.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.4.1 – §7.4.2.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §4.10–2.

4.4 Improper Integrals

For our present purposes, an integral will be “improper” if it has any of the following problems:

- its integrand goes to a finite limiting value at finite upper and lower limits, but cannot be evaluated *right on* one of those limits (e.g., $\sin x/x$ at $x = 0$)
- its upper limit is ∞ , or its lower limit is $-\infty$
- it has an integrable singularity at either limit (e.g., $x^{-1/2}$ at $x = 0$)
- it has an integrable singularity at a known place between its upper and lower limits
- it has an integrable singularity at an unknown place between its upper and lower limits

If an integral is infinite (e.g., $\int_1^\infty x^{-1} dx$), or does not exist in a limiting sense (e.g., $\int_{-\infty}^\infty \cos x dx$), we do not call it improper; we call it impossible. No amount of clever algorithmics will return a meaningful answer to an ill-posed problem.

In this section we will generalize the techniques of the preceding two sections to cover the first four problems on the above list. A more advanced discussion of quadrature with integrable singularities occurs in Chapter 19, notably §19.3. The fifth problem, singularity at an unknown location, can really only be handled by the use of a variable stepsize differential equation integration routine, as will be given in Chapter 17, or an adaptive quadrature routine such as in §4.7.

We need a workhorse like the extended trapezoidal rule (equation 4.1.11), but one that is an *open* formula in the sense of §4.1, i.e., does not require the integrand to be evaluated at the endpoints. Equation (4.1.19), the extended midpoint rule, is the best choice. The reason is that (4.1.19) shares with (4.1.11) the “deep” property of having an error series that is entirely even in h . Indeed there is a formula, not as well known as it ought to be, called the *Second Euler-Maclaurin summation formula*,

$$\begin{aligned}
 \int_{x_0}^{x_{N-1}} f(x) dx &= h[f_{1/2} + f_{3/2} + f_{5/2} + \cdots + f_{N-5/2} + f_{N-3/2}] \\
 &+ \frac{B_2 h^2}{4} (f'_{N-1} - f'_0) + \cdots \\
 &+ \frac{B_{2k} h^{2k}}{(2k)!} (1 - 2^{-2k+1}) (f_{N-1}^{(2k-1)} - f_0^{(2k-1)}) + \cdots
 \end{aligned} \tag{4.4.1}$$

This equation can be derived by writing out (4.2.1) with stepsize h , then writing it out again with stepsize $h/2$, and then subtracting the first from twice the second.

It is not possible to double the number of steps in the extended midpoint rule and still have the benefit of previous function evaluations (try it!). However, it is possible to *triple* the number of steps and do so. Shall we do this, or double and accept the loss? On the average, tripling does a factor $\sqrt{3}$ of unnecessary work, since the “right” number of steps for a desired accuracy criterion may in fact fall anywhere in the logarithmic interval implied by tripling. For doubling, the factor is only $\sqrt{2}$, but we lose an extra factor of 2 in being unable to use all the previous evaluations. Since $1.732 < 2 \times 1.414$, it is better to triple.

Here is the resulting structure, which is directly comparable to Trapzd.

quadrature.h

```
template <class T>
struct Midpnt : Quadrature {
    Routine implementing the extended midpoint rule.
    Doub a,b,s;                      Limits of integration and current value of inte-
    T &func;                          gral.
    Midpnt(T &func, const Doub aa, const Doub bb) :
        funk(func), a(aa), b(bb) {n=0;}
        The constructor takes as inputs func, the function or functor to be integrated between
        limits a and b, also input.
    Doub next(){
        Returns the nth stage of refinement of the extended midpoint rule. On the first call (n=1),
        the routine returns the crudest estimate of  $\int_a^b f(x)dx$ . Subsequent calls set n=2,3,... and
        improve the accuracy by adding  $(2/3) \times 3^{n-1}$  additional interior points.
        Int it,j;
        Doub x,tnm,sum,del,ddel;
        n++;
        if (n == 1) {
            return (s=(b-a)*func(0.5*(a+b)));
        } else {
            for(it=1,j=1;j<n-1;j++) it *= 3;
            tnm=it;
            del=(b-a)/(3.0*tnm);
            ddel=del+del;
            x=a+0.5*del;
            sum=0.0;
            for (j=0;j<it;j++) {
                sum += func(x);
                x += ddel;
                sum += func(x);
                x += del;
            }
            s=(s+(b-a)*sum/tnm)/3.0;
            return s;
        }
    }
    virtual Doub func(const Doub x) {return funk(x);} Identity mapping.
};
```

You may have spotted a seemingly unnecessary extra level of indirection in Midpnt, namely its calling the user-supplied function funk through an identity function func. The reason for this is that we are going to use mappings other than the identity mapping between funk and func to solve the problems of improper integrals listed above. The new quadratures will simply be derived from Midpnt with func overridden.

The structure Midpnt could be used to exactly replace Trapzd in a driver routine like qtrap (§4.2); one could simply change Trapzd<T> t(func,a,b) to Midpnt<T> t(func,a,b), and perhaps also decrease the parameter JMAX since

$3^{J_{\text{MAX}}-1}$ (from step tripling) is a much larger number than $2^{J_{\text{MAX}}-1}$ (step doubling). The open formula implementation analogous to Simpson's rule (qsimp in §4.2) could also substitute Midpnt for Trapzd, decreasing JMAX as above, but now also changing the extrapolation step to be

```
s=(9.0*st-ost)/8.0;
```

since, when the number of steps is tripled, the error decreases to 1/9th its size, not 1/4th as with step doubling.

Either the thus modified qtrap or qsimp will fix the first problem on the list at the beginning of this section. More sophisticated, and allowing us to fix more problems, is to generalize Romberg integration in like manner:

```
template<class T> romberg.h
Doub qromo(Midpnt<T> &q, const Doub eps=3.0e-9) {
    Romberg integration on an open interval. Returns the integral of a function using any specified
    elementary quadrature algorithm q and Romberg's method. Normally q will be an open formula,
    not evaluating the function at the endpoints. It is assumed that q triples the number of steps
    on each call, and that its error series contains only even powers of the number of steps. The
    routines midpnt, midinf, midsq1, midsqu, midexp are possible choices for q. The constants
    below have the same meanings as in qromb.
    const Int JMAX=14, JMAXP=JMAX+1, K=5;
    VecDoub h(JMAXP),s(JMAX);
    Poly_interp polint(h,s,K);
    h[0]=1.0;
    for (Int j=1;j<=JMAX;j++) {
        s[j-1]=q.next();
        if (j >= K) {
            Doub ss=polint.rawinterp(j-K,0.0);
            if (abs(polint.dy) <= eps*abs(ss)) return ss;
        }
        h[j]=h[j-1]/9.0; This is where the assumption of step tripling and an even
                                                                    error series is used.
    }
    throw("Too many steps in routine qromo");
}
```

Notice that we now pass a Midpnt object instead of the user function and limits of integration. There is a good reason for this, as we will see below. It does, however, mean that you have to bind things together before calling qromo, something like this, where we integrate from a to b:

```
Midpnt<Ftor> q(ftor,a,b);
Doub integral=qromo(q);
```

or, for a bare function,

```
Midpnt<Doub(Doub)> q(fbare,a,b);
Doub integral=qromo(q);
```

Laid back C++ compilers will let you condense these to

```
Doub integral = qromo(Midpnt<Ftor>(Ftor()),a,b));
```

or

```
Doub integral = qromo(Midpnt<Doub(Doub)>(fbare,a,b));
```

but uptight compilers may object to the way that a temporary is passed by reference, in which case use the two-line forms above.

As we shall now see, the function qromo, with its peculiar interface, is an excellent driver routine for solving all the other problems of improper integrals in our first list (except the intractable fifth).